COSC 1030 Physical Computing Project 2: Buttons and multiple processes

Introduction: Physical computing gives us the opportunity to introduce elements of parallelization into a program. In this program we are going to learn to use a button as a digital sensor that will control which process is running.

Equipment Needed:

- Raspberry Pi
- 80 ohm - 1K ohm resistor
- An LED of any color
- Button x 2
- Breadboard
- Jumper wires

(1) First set up and test a circuit and program that can control an LED as in Project 1.

(2) Now add a button to your breadboard as described here and test it out. Note that the gpiozero Button class automatically enables a "pull-up" resistor that is part of the Pi circuitry, so we don't need a resistor in this circuit.

(3) Now take a look at the following code. Here we have two functions that are called, one when the button is pressed and one when the button is released. Try out this code (note this is set up to have the button connected to GPIO17, you might have to change it if your button is not connected to this GPIO pin). This code sets the function to call when the `button` object is pressed (`when_pressed`) to the function `say_hello()` and sets the function to call `when_released` to `say_goodbye()`. The call to `pause()` at the end is needed because this is an **event driven program**. This tells python to keep the script running and wait for something to happen.

```
from gpiozero import Button
from signal import pause

button = Button(17)

def say_hello():
    print("Button was pressed!")


def say_goodbye():
    print("Button was released")

button.when_pressed = say_hello
button.when_released = say_goodbye

pause()
```

Take a look at the documentation for the gpiozero package. What other function can be set to call some function when an action happens?

(4) Change this program so that when you press the button the LED turns on and when you release the button the LED turns off.

(5) Now change the program so that when you press the button, LED will transmit a one or more character message in Morse Code. (Make a long flash for a dash and a short flash for a dot.) The message should keep repeating once the button is pressed.

(6) Now we would like to change this program so that when the button is pressed the Morse code message starts transmitting and when we hit the button again, it stops transmitting. This is going to require us to use two processes, one that is monitoring for a button press and another that is managing the repeated message transmission. To manage these processes we will need the `multiprocessing` package. Take a look at the following code.

```
import multiprocessing
from time import sleep
from gpiozero import LED, Button
from signal import pause

red = LED(14)
button = Button(17)
process = None


def message():
    # code to transmit the Morse Code message
    while True:
     red.on()
     sleep(1)
     red.off()
     sleep(1)

def button_callback():
    global process
    print("Button  pressed")

    if process and process.is_alive():
        print("stopping message")
        process.terminate()
        process.join()
        red.off()
    else:
        print("launching message")
        process = multiprocessing.Process(target=message)
        process.start()

button.when_pressed = button_callback

print("Ready to run")
pause()
```

The function that is called when the button is pressed (button_callback) has some new complications to it. First notice the `global process`, this is going to keep track of the process that is running the `message` function. The `else` part of the decision structure starts the child process that is going to be in charge of the message, this means that this part of the program is going to be run on a different CPU core at the same time as the main process is waiting for a button press. The `target=message` part of the code says that the function that will be run on another core will be the `message` function

defined above in this file. The `if` part of the decision structure is checking to see if the process is currently running and if it is, the lines underneath stop the process and join back with the main process that will still be monitoring the button press.

Change the message function to be the function you wrote to do your Morse code message and get this working.

(7) Using a circuit that has two buttons and an LED, write a program that will allow button 1 to make the LED send message 1 and a button press to button 2 will change the message that the LED is transmitting to message 2.

TURN IN: A video tour of the code and demonstration of it running.